Agent Foundations — Write-Up

Week 2, Day 1 · Reasoning Loops, Tool Calling & Raw Python Agents (built with the Gemini API)

The ReAct Loop

The agent is a plain while-loop, not a framework. Each turn: the model is sent the running message history and reasons about what to do (Reason); if it decides more information is needed, it emits a function_call step naming a tool and arguments (Act); the harness executes that tool locally and appends the result back into the history as a function_result (Observe); the loop repeats until the model returns plain text instead of a tool call, which is treated as the final answer. A max_iterations cap guarantees the loop terminates even if the model never stops calling tools.

```
Reason → Act (call tool) → Observe (tool result) → repeat → final answer
```

Tool Schemas Used

- calculator(expression: string) — evaluates a single arithmetic expression (e.g. "(15 * 4) + 7") using ast.parse and a whitelist of operators; explicitly does not handle word problems or units.
- get_weather(city: string) — stub lookup returning temperature and conditions for a small fixed set of cities; raises a clear error for any city outside that set.
- read_text_file(path: string) — reads and returns the contents of a local text file.

Each schema pairs a name with a description written for the model, not the developer — the description is the model's entire understanding of when and how to use the tool, since it never sees the implementation. Specific descriptions with an example input and an explicit boundary case reduce two failure modes: the model skipping the tool and hallucinating an answer, or calling the wrong tool for the job.

Failure Modes Observed

The agent was deliberately broken four ways: an ambiguous request, a tool call outside its stub data, a task requiring an undefined tool, and a forced runaway loop. Observed behavior and mitigations:

Failure Mode	What Happens	Mitigation
Ambiguous request	Model asks a clarifying question instead of guessing (e.g. asks which city).	System prompt instructs the model to only call a tool when it has enough information.
Tool/data error	Requesting data outside the stub dataset raises an exception inside the tool.	execute_tool() catches exceptions and returns an error string as the tool result, so the model explains the failure instead of fabricating data.
Hallucinated / undefined tool call	Model may invent a tool that was never registered (e.g. send_email).	execute_tool() checks the name against the registered tool list first and returns a clear "unknown tool" message.
Wrong or malformed arguments	Model passes badly-formed input (e.g. a word problem instead of an expression).	calculator() parses with ast.parse and an operator whitelist (no eval()), so bad input raises a caught error.
Infinite / runaway loop	A model that keeps calling tools never reaches a final text-only turn.	Hard max_iterations cap in run_agent() stops execution with a guardrail log line.
Silent errors	Any of the above, uncaught, could crash the process or let the model reason from garbage.	Every failure path returns an explicit, model-visible error string, so both the model and the developer (via logs) know something went wrong.

Why Frameworks Exist

The full working agent is roughly 150 lines: tool schemas, a dispatch dictionary, an execute_tool wrapper with error handling, and a while-loop with a logging trace and an iteration cap. None of it is magic — which is the point of building it by hand first. At scale, every piece here grows: dozens of tools instead of three, multi-agent handoffs, streaming responses, retries with backoff, persistent cross-session memory, parallel tool execution, and human-in-the-loop approval steps. Frameworks like LangChain, LangGraph, and CrewAI exist to standardize those recurring pieces so teams aren't rewriting the same loop and the same try/except around every tool call in every project. Having built the loop by hand first means their abstractions read as named versions of things already understood, which is what makes debugging a framework-based agent tractable when it misbehaves.